

GeoLabeller

Interface Control Document

Software version v1.8.3

Generated 2026-09-08

This document specifies the external interfaces of GeoLabeller: the project file it reads and writes, the raster products it exports, the imagery it accepts, and the coordinate conventions all of these share. It is a description of the implemented behaviour, transcribed from the source.

1 Scope and Conventions

GeoLabeller is a desktop application for viewing georeferenced and non-georeferenced raster imagery and recording point annotations against it. This document specifies the interfaces through which other software exchanges data with it.

1.1 Interfaces covered

Interface	Direction	Section
Project file (.geoproj / JSON)	Read and written	3
Sub-image GeoTIFF export	Written	4.1
Optimized GeoTIFF export	Written	4.2
Source imagery	Read	5

The HDF5 training-dataset export is specified separately and is not repeated here.

This document ships with the application: an installed copy sits beside the executable and opens from Help > ICD, so the version on hand always describes the build in front of you.

1.2 Notation

- Field types are JSON types: number, string, boolean, array, object.
- Pixel coordinates are (column, row) with the origin at the top-left pixel of the source image; column increases right, row increases down. Fractional values are meaningful and are not rounded on write.
- Geographic coordinates are WGS84 decimal degrees (EPSG:4326), longitude and latitude reported separately and never packed.
- Distances in metres are geodesic (Haversine on WGS84), not planar distances in a projected CRS.

2 Coordinate Systems

Four coordinate systems appear across the interfaces. Only the first two are externally visible; the rest are described because they explain values that would otherwise look surprising.

System	Units	Where it appears
WGS84 (EPSG:4326)	degrees	Label lon/lat, waypoints, image corner coordinates
Source pixel	pixels	Label pixel_x / pixel_y, relative to the unmodified source image
Web Mercator (EPSG:3857)	metres	Internal display projection; not written to any file
Scene	metres	Internal canvas space; x = easting, y = -northing

2.1 Pixel coordinates are relative to the source image

Labels record the pixel they were placed on in the ORIGINAL image, at full resolution - not in the reprojected image shown on screen, and not at whatever pyramid level happened to be displayed. A label therefore addresses the same ground whatever the zoom was when it was placed, and a consumer can index the source raster directly.

2.2 Non-georeferenced imagery: the pixel zone

Imagery without a CRS cannot be placed geographically, so it is laid out in a reserved region of the internal scene beyond the valid extent of Web Mercator. This is an implementation detail with one external consequence: labels on such images carry pixel coordinates but no meaningful lon/lat, and those fields are written as 0.0.

Constant	Value	Meaning
PIXEL_ZONE_ORIGIN_X	25 000 000	Scene X at which the non-georeferenced region begins
PIXEL_ZONE_SCALE	50	Scene units per source pixel
PIXEL_ZONE_GROUP_GAP	5 000	Scene units between the columns of adjacent groups

2.3 Display resolution and memory

Imagery is displayed from pyramid overviews. The level chosen is the coarsest whose resolution is still no coarser than one screen pixel, subject to a cap on how much may be held at once. Images beyond that cap keep a coarse whole-image backdrop and read the detail for the visible area one tile at a time, so full resolution remains available at any zoom regardless of image size.

Constant	Value	Meaning
MAX_LEVEL_PIXELS	150 000 000	Largest whole-image level held in memory
BACKDROP_MAX_PIXELS	4 000 000	Cap for the backdrop behind windowed detail tiles
TILE_SIZE	512	Edge of a display tile, in destination pixels

3 Project File

A project is a single UTF-8 JSON document holding the class list, every image referenced, the labels on them, and any waypoints. Images are referenced by absolute path; their pixels are never copied into the project.

3.1 Top level

Field	Type	Description
version	string	Format version of this document, currently "4.0". Governs how label coordinates are interpreted - see 3.4.
classes	array of string	Ordered class names. A label's class must appear here.
images	array of object	One entry per referenced image; see 3.2.
waypoints	array of object	Named geographic bookmarks; see 3.5. Absent before version 3.3.
_next_id	number	Next label id to allocate. Labels ids are unique per project.
_next_waypoint_id	number	Next waypoint id to allocate.

3.2 Image object

Field	Type	Description
path	string	Absolute path to the image file. On load, a path that does not exist is first re-tried against the project file's own folder by matching progressively longer path tails, so a project that travels with its imagery opens unchanged on another machine; anything still missing can be relocated interactively (File > Locate Missing Images).
name	string	File name without extension.
group	string	Group hierarchy, '/' separated; empty when ungrouped.
labels	array of object	Labels on this image; see 3.3.
original_width	number	Source image width in pixels. Required to interpret label coordinates - see 3.4.
original_height	number	Source image height in pixels.
reader	object	Extension to reader name, e.g. {"tif": "default"}.
affine_coeffs	array of number	Six coefficients of the source affine transform. Omitted when unknown.
crs_epsg	number	EPSG code of the source CRS. Omitted for non-georeferenced imagery.
corners_wgs84	object	Convenience only: top_left / top_right / bottom_right / bottom_left, each {lat, lon}. Written when the transform and CRS are known; derived, never read back.
geodesic_width_m	number	Convenience only: ground width along the top edge, metres.
geodesic_height_m	number	Convenience only: ground height along the left edge, metres.
hard_negative_source	boolean	The image contains confusers but no true positives; the H5 export can opt in to sliding it whole into hard negatives. Present only when true; absent before version 3.5. Kept even while the image is not loaded, like labels.

3.3 Label object

Field	Type	Description
id	number	Sequential id, unique within the project.
unique_id	string	UUID v4, unique to this label.
class_name	string	Must appear in the top-level classes.
pixel_x	number	Column in the source image. Stored as a FRACTION of original_width from version 2.1 - see 3.4.

Field	Type	Description
pixel_y	number	Row, stored the same way.
lon	number	WGS84 longitude, or 0.0 for non-georeferenced imagery.
lat	number	WGS84 latitude, or 0.0.
object_id	string	UUID v4 shared by every label of the same real-world object. Labels are linked by sharing this value.
length_m	number	Measured object length in metres. Omitted when unmeasured.
width_m	number	Measured width in metres. Omitted likewise.
description	string	Free text describing this labelled object. Belongs to this label alone and is never copied to labels sharing its object_id. Omitted when empty; absent before version 3.4.
orientation_px_rad	number	Unit-circle principal angle of the object's drawn orientation in UN-WARPED source-pixel space: radians in $(-\pi, \pi]$, x along the rows (+column), y up the columns. A vector from top-right to bottom-left is about $-3\pi/4$ (-135 degrees); rightward along a row is 0; up the image is $+\pi/2$. Omitted when never drawn; absent before 3.7.
orientation_deg	number	The same drawn vector as a TRUE-north heading: degrees clockwise from north in $[0, 360)$, from a WGS84 geodesic azimuth of the endpoints through the image's transform - not grid north. Omitted for non-georeferenced imagery or when never drawn. Per label even when linked: two views of one object are two measurements.
orientation_derived	boolean	Present (true) only when this orientation was NOT drawn by hand but propagated from a linked label: the group's shared true-north heading, re-derived into this image's own pixel space through its georeferencing. Cleared when the user draws over it. Absent before 3.8.
masks	array of object	Named binary masks painted on this label's snippet in the mask editor, for measuring the object's pixel distribution against the background. Each entry is {name, x0, y0, width, height, rle}; the full encoding is specified in 3.8. From version 4.0 the window is the full source image. Present only when at least one mask exists; absent before 3.9.
group_id	string	Human-readable name for the linked-object group - the readable companion to object_id. The inverse contract to description: it SHALL be identical across every label sharing an object_id. Updating it anywhere updates the whole group; a file that violates the invariant is repaired on load (first non-empty name in the group wins). Omitted when empty; absent before 3.6.
geodesic_distance	number	Convenience only: metres from the image's left edge at the label's row. Derived, never read back.

3.4 Label coordinate storage (important)

From version 2.1, pixel_x and pixel_y are stored as FRACTIONS of the image dimensions, not as absolute pixels. A consumer must multiply by the image size to recover pixel coordinates:

```
pixel_x_absolute = pixel_x * original_width
pixel_y_absolute = pixel_y * original_height
```

The conversion applies only when the document version is 2.1 or later AND original_width and original_height are both greater than zero. If either is zero the values are absolute pixels and must be used unchanged. Both the writer and the reader apply this rule, so a file is self-consistent; it matters when reading a project with anything other than GeoLabeller itself.

3.5 Waypoint object

Waypoints are named geographic bookmarks belonging to the project rather than to any image. They are navigation aids and are never exported as training data.

Field	Type	Description
id	number	Sequential id, unique within the project.
name	string	Display name, e.g. "WP 1".
lat	number	WGS84 latitude in degrees.
lon	number	WGS84 longitude in degrees.

3.6 Version history

Version	Change
1.0	Label-centric layout: a flat labels array, each entry naming its image. Still readable.
2.0	Image-centric layout: images each own their labels.
2.1	Label pixel coordinates became fractions of the image dimensions (see 3.4).
3.2	Adds derived convenience fields: corner coordinates and geodesic extents.
3.3	Adds the top-level waypoints array and <code>_next_waypoint_id</code> . Files written before this simply have none.
3.4	Adds the optional per-label description string.
3.5	Adds the optional per-image <code>hard_negative_source</code> flag.
3.6	Adds the shared per-group <code>group_id</code> string on labels.
3.7	Adds the optional <code>orientation_px_rad</code> and <code>orientation_deg</code> per label.
3.8	Adds the optional <code>orientation_derived</code> flag for orientations propagated from a linked label.
3.9	Adds the optional per-label masks array of named, run-length-encoded binary snippet masks (see 3.8).
4.0	Mask entries are written anchored to the FULL source image (<code>x0 = y0 = 0</code> , width/height = the image's) instead of the paint-time snippet window. The schema is unchanged and readers honor whichever window an entry records; 3.9-windowed entries are re-anchored on load. Numbered 4.0 rather than 3.10 because versions compare as strings and "3.10" would sort below "3.9".

Readers should compare the version as a string and treat anything at or above a threshold as supporting that feature. Older documents load unchanged; the only migration is the 4.0 mask re-anchoring, which the reader performs structurally on load.

3.7 Example

A project with one image, one label and one waypoint. Note the fractional pixel coordinates: label 7 sits at absolute pixel (2048, 1024), being 0.5 x 4096 and 0.25 x 4096.

```
{
  "version": "3.4",
  "classes": ["vehicle", "building"],
  "images": [
    {
      "path": "D:/imagery/tile_014.tif",
      "name": "tile_014", "group": "survey/north",
      "original_width": 4096, "original_height": 4096,
      "reader": {"tif": "default"}, "crs_epsg": 32618,
      "labels": [
        {
          "id": 7, "class_name": "vehicle",
          "pixel_x": 0.5, "pixel_y": 0.25,
          "lon": -73.9832, "lat": 40.7536,
          "unique_id": "3b1e...", "object_id": "8f2c...",
          "length_m": 4.6, "width_m": 1.8,
          "description": "white pickup, tarpaulin over the bed"
        }
      ]
    }
  ],
  "waypoints": [{"id": 1, "name": "WP 1",
    "lat": 40.7536, "lon": -73.9832}],
  "_next_id": 8, "_next_waypoint_id": 2
}
```

3.8 Snippet masks and their encoding

A mask marks which pixels of a label's snippet belong to the actual object. Masks are stored per label as independent named binary layers - several may coexist and OVERLAP on one snippet - and are painted in the mask editor (Labels > Mask Editor).

Anchoring. x0/y0/width/height describe the mask's window in SOURCE-image pixels. From format 4.0 the window written IS the full source image - x0 and y0 are 0 and width/height equal the image dimensions - so the decoded rle is co-registered with the imagery: mask pixel (row, col) is image pixel (row, col), with no anchoring arithmetic. Format 3.9 wrote the snippet crop at the moment the mask was painted instead (per the centred-window rule of 4.1). Readers SHALL honor whichever window an entry records - a mask re-projects into any other window, a different snippet size or an export crop, by pure translation and clipping - and SHOULD stream the runs rather than decode the full window, which for a 4.0 entry over survey imagery can cover billions of pixels. Only the runs bordering unpainted ground grow with the image; the run COUNT is essentially the snippet-window count.

Compression. The rle field run-length encodes the binary window read row-major (row 0 left to right, then row 1, and so on). Entries alternate between runs of 0s and runs of 1s, and the list ALWAYS begins with a run of 0s - a mask whose very first pixel is set therefore begins [0, n, ...]. The runs MUST sum to exactly width x height; readers SHALL reject any mask where they do not, rather than render a shifted mask. Worked example: the 4x3 window

```
0 0 1 1
0 0 1 1
0 0 0 0

encodes as rle = "2,2,2,2,4"
(2 zeros, 2 ones, 2 zeros, 2 ones, 4 zeros = 12 pixels)
```

A painted blob of a few thousand pixels typically encodes to a few hundred integers, which is why masks can live inside the project JSON and share its save/autosave/recovery behaviour rather than requiring a sidecar file.

Delivery (informative). The HDF5 training export - specified separately, see 1.1 - carries the painted masks in three aligned datasets: mask_pixels (M x H x W, uint8 binary), mask_sample (M, the images row each mask belongs to) and mask_names (M, UTF-8). Every example crop receives every mask that intersects it, re-anchored to that crop's window - so in the HDF5 there is no x0/y0 to apply: mask_pixels[j] overlays images[mask_sample[j]] pixel for pixel as-is. Extracting them needs nothing beyond h5py:

```
with h5py.File(path) as f:
    for j in range(f['mask_pixels'].shape[0]):
        image = f['images'][f['mask_sample'][j]]
        mask = f['mask_pixels'][j].astype(bool)
        name = f['mask_names'][j].decode()
        object_pixels = image[mask]
```


4 Raster Exports

4.1 Sub-image export

Writes one GeoTIFF per label, cropped around it. The requested size is given in ground metres and converted to pixels using the true resolution at the label, so the crop covers the requested ground area whatever the CRS and whatever the pixel aspect.

Property	Behaviour
Output path	<output>/<class_name>/<object_id>_<label_id padded to 6>.tif
Window	Centred on the label's pixel, shifted (never cropped) to stay inside the image, so edge labels still yield a full-size crop.
Pixels	Copied unmodified: all source bands, original data type, no rescaling.
Georeferencing	Source CRS preserved; transform adjusted to the window.
Other metadata	nodata and per-band colour interpretation preserved.
Compression	LZW.
Skipped	Labels on imagery with no CRS, labels outside the image, and windows that would be under half the requested size.

4.2 Optimized GeoTIFF export

Rewrites imagery as tiled, pyramided GeoTIFFs for fast display. Pixel values, band count, data type, CRS and transform are all preserved - only the storage layout changes.

Property	Behaviour
Output path	Mirrors the group hierarchy under the output directory; keeps the source stem with a .tif extension.
Layout	Tiled, 512 x 512 blocks by default.
Overviews	Built to the requested decimation factors. Average resampling by default; paletted rasters always use nearest, so palette indices are never blended into invalid values.
Compression	DEFLATE by default; may be set to another codec, none, or KEEP to retain the source's. A predictor is chosen to suit the data type.
Large files	BIGTIFF written when the result may exceed 4 GB.
Existing files	Skipped unless overwrite is requested. Output is written to a temporary file and moved into place, so an interrupted export leaves no partial .tif.

5 Source Imagery

Imagery is read through GDAL, so any format GDAL can open may be loaded; file dialogs filter to .tif and .tiff by default, and directory import scans for those extensions.

5.1 Georeferenced and non-georeferenced imagery

An image with a CRS is reprojected for display and its labels carry both pixel and WGS84 coordinates. An image without one is shown at its own pixel scale in the region described in 2.2, and its labels carry pixel coordinates only, with lon and lat written as 0.0.

5.2 Recommendations

- Supply tiled GeoTIFFs with overviews. Without a pyramid the whole image must be decoded before anything can be shown; with one, a coarse level appears immediately and sharpens. Section 4.2 produces suitable files.
- 8-bit imagery is displayed exactly as stored. Other data types are linearly stretched for display between the 2nd and 98th percentile of a sampled read; the underlying values are never modified.
- Very large mosaics are supported: detail for the visible area is read a tile at a time, so display cost follows the viewport rather than the size of the image.

6 Operation

This section is included because the way a label is produced decides which fields it carries. A consumer reading a project file will meet labels that share an `object_id`, labels with measurements and labels whose lon/lat are zero, and 6.3 explains where each comes from.

6.1 Interaction modes

The canvas is always in exactly one mode. Modes differ in what a left click does and how the view is navigated; none of them change how a label is stored.

Mode	Left click	Purpose
Pan	Drags the view	Navigating. Right click also offers waypoint and layer actions.
Label	Places a label	Ordinary labelling of whatever is on screen.
Cycle	Places a label	Steps through a group one image at a time; Space advances.
View Cycle	Places a label	As Cycle, but over the images currently in view.
Waterfall	Places a label	Stacks a bottom-level group vertically as raw pixels and glides through it; every label in the group stays visible. Leaving the mode returns the view to the geography of the image being examined.
Ruler	Drags a measurement	Reading ground distances. Records nothing in the project.

Two overlays work on top of whichever labelling mode is active, rather than replacing it:

Overlay	Effect
Chain Link	Clicking labels one after another links them into a single object; each click writes the shared <code>object_id</code> immediately. Right-clicking a label also offers Link by Box: drag a desktop-style box and every label inside is linked to that label's object. Linked groups always wear a coloured halo ring - one colour per group, distinct between groups, updated the moment links change.
Measure	Draws a length and then a width line on a label, writing <code>length_m</code> and <code>width_m</code> . Needs source georeferencing, since metres are undefined without a CRS - but works in waterfall mode too: the drawn line is mapped through source pixels back to the ground, so stacked and geographic measurements agree.
Description	Right click a label to write a free-text note on it, shown as the marker's tooltip. Stays with that label when it is linked.
Locate missing images	File menu. Rebinds a shared project's images to their paths on this machine: search a folder (filename matches verified against each image's recorded size and CRS) or locate one file and apply its old-versus-new path prefix to the rest. Applied paths persist when the project is saved.
Orient	Labels > Orientation Editor shows a class's snippets in a grid; dragging tail-to-nose across one stores <code>orientation_px_rad</code> and, for georeferenced imagery, <code>orientation_deg</code> . Snippets are raw source pixels with the export's framing, so the drawn vector is a source-pixel vector. Oriented snippets get a coloured border: amber for hand-drawn, violet for propagated. With the 'apply heading to linked labels' toggle on, drawing on a linked label gives the group's other labels the same true-north heading, each with a pixel angle derived through its own image (marked <code>orientation_derived</code> until drawn over by hand).
Mask	Labels > Mask Editor paints named binary masks on a label's snippet (left-drag paints, right-drag erases; masks may overlap). The snippet size is selectable; masks are stored anchored to the full source image, so resizing changes only the window being painted - existing masks are never moved or cropped by it. The wheel zooms to the cursor (0.25x-32x, pixels stay crisp) and Shift+drag pans, so single-pixel edits are practical. Space / Ctrl+Space step through the snippet strip, filterable to one class or all. Fill Enclosed floods the interior of a drawn outline, and refuses when the outline has a gap (nothing enclosed). A live readout compares the object's RAW per-band mean/std against the background. Stored per 3.8; carried into the HDF5 export automatically.
Flag hard negatives	Right click an image to mark it as a hard-negative source: it holds confusers but no true positives. Flagged images collect in the Hard Negatives panel, and the H5 export can include them as <code>gt=false</code> hard negatives even under a labels-only scope.

6.2 How a label's fields are produced

Every field in 3.3 is written for every label; these are the ones whose value depends on how or where the label was made.

Field	Produced by	Consumer note
pixel_x, pixel_y	Any left click in a labelling mode	Always the source image's own pixels at full resolution, whatever the zoom or pyramid level on screen. Stored as fractions - see 3.4.
lon, lat	Derived from the clicked pixel through the image's CRS	Zero when the image has no CRS. Check crs_epsg on the image before trusting these.
object_id	Shared when labels are linked	Unique per label until then. Labels of one real object are found by grouping on this value, not by proximity.
length_m, width_m	Measure overlay	Absent when never measured. Geodesic metres, not planar. Copied to every label sharing the object_id when the linked-measurement option is on.
description	Right click a label, Description...	Free text, absent when empty. Per label even when labels are linked: the same object in two images is described separately.
group_id	Right click a label, Group ID...	One value for the whole linked group, always: setting it on any member applies it to every label sharing the object_id. A label that is unlinked loses it; a label joining a named group adopts the name (the first label's name wins a merge conflict).
class_name	The class selected when the click was made	Always one of the top-level classes.

6.3 Keyboard and mouse reference

Generated from the application's own binding table, so it matches the reference shown by F1 exactly.

File Operations	
Ctrl+N	New Project
Ctrl+Shift+P	Open Project
Ctrl+S	Save Project
Ctrl+Shift+S	Save Project As
Ctrl+O	Add Image (GeoTIFF + custom)
Ctrl+Shift+O	Add Directory
Ctrl+Q	Exit

Navigation	
Mouse Wheel	Zoom in/out
Click + Drag	Pan (in Pan mode)
Right-click	Context menu
Ctrl+G	Go to a latitude/longitude
Ctrl+Shift+W	Add waypoint by coordinates

Mode Switching	
P	Pan mode
L	Label mode
C	Cycle mode (group-based)
V	View Cycle (layers in view)

Mode Switching	
W	Waterfall mode (stack group vertically)
R	Ruler mode

Labeling (Label & Cycle modes)	
Left-click	Place label
Right-click label	Label options (remove, link, measure)
Right-click + drag	Pan the view
Ctrl+Left-click label	Label options (Cycle modes)
1-9	Quick-switch to class 1-9
Escape	Cancel link mode

Chain Linking	
K	Toggle chain-link (any labeling mode)
Left-click label	Add to chain (first click anchors)
N	Finish this chain, start a new one
Escape	Exit chain-link (links are kept)

Cycle Modes (Cycle / View Cycle)	
Space	Next layer (unchecks current)
Ctrl+Space	Go back to previous layer
Left-click	Place label
Right-click label	Label options (remove, link, measure)
Right-click + drag	Pan around
Mouse wheel	Zoom in/out

Waterfall Mode	
Hold Space	Glide up through the image stack
Hold Ctrl+Space	Glide down (starts at the bottom)
Left-click	Place label (group labels stay visible)
Right-click label	Label options (remove, link, measure)
Right-click + drag	Pan within the stack
Mouse wheel	Zoom in/out

Measuring	
Shift + Drag	Measure distance, without changing mode
M	Measure label under cursor (length, width)
Escape	Clear the measurement or the go-to marker

Waypoints	
Right-click map	Add waypoint here (Pan mode, geo)
Ctrl+Shift+W	Add waypoint by coordinate
Right-click marker	Go to / Rename / Remove
Double-click in panel	Fly to that waypoint
Show on map	Hide or show every waypoint marker

Layer Panel	
Checkbox	Toggle layer/group visibility
Right-click group	Select/Unselect, Expand/Collapse all
Right-click layer	Zoom to layer, Remove
Drag & Drop	Reorder layers/groups

Labeled Images Panel	
Checkbox	Toggle image visibility (synced)
Right-click label	Zoom to label or layer
Right-click group	Select/Unselect all in group

Help	
F1	Show this help